Key Technologies

PostgreSQL

Learn when and how to use PostgreSQL in your system design interviews

There's a good chance you'll find yourself discussing PostgreSQL in your system design interview. After all, it's consistently ranked as the most beloved database in **Stack Overflow's developer survey** and is used by companies from Reddit to Instagram and even the very website you're reading right now.

That said, it's important to understand that while PostgreSQL is packed with features and capabilities, your interviewer isn't looking for a database administrator. They want to see that you can make informed architectural decisions. When should you choose PostgreSQL? When should you look elsewhere? What are the key trade-offs to consider?

I often see candidates get tripped up here. They either dive too deep into PostgreSQL internals (talking about MVCC and WAL when the interviewer just wants to know if it can handle their data relationships), or they make overly broad statements like "NoSQL scales better than PostgreSQL" without understanding the nuances.

In this deep dive, we'll focus specifically on what you need to know about PostgreSQL for system design interviews. We'll start with a practical example, explore the key capabilities and limits that should inform your choices, and build up to common interview scenarios.

For this deep dive, we're going to assume you have a basic understanding of SQL. If you don't, I've added an **Appendix: Basic SQL Concepts** at the end of this page for you to review.

Let's get started.

A Motivating Example

Let's build up our intuition about PostgreSQL through a concrete example. Imagine we're designing a social media platform - not a massive one like Facebook, but one that's growing and needs a solid foundation.

Our platform needs to handle some fundamental relationships:

- Users can create posts
- Users can comment on posts
- Users can follow other users
- Users can like both posts and comments
- Users can create direct messages (DMs) with other users

> This is exactly the kind of scenario that comes up in interviews. The relationships between entities are clear but non-trivial, and there are interesting questions about data consistency and scaling.

What makes this interesting from a database perspective? Well, different operations have different requirements:

- Multi-step operations like creating DM threads need to be atomic (creating the thread, adding participants, and storing the first message must happen together)
- Comment and follow relationships need referential integrity (you can't have a comment without a valid post or follow a non-existent user)
- Like counts can be eventually consistent (it's not critical if it takes a few seconds to update)
- When someone requests a user's profile, we need to efficiently fetch their recent posts, follower count, and other metadata
- Users need to be able to search through posts and find other users
- As our platform grows, we'll need to handle more data and more complex queries

This combination of requirements - complex relationships, mixed consistency needs, search capabilities, and room for growth - makes it a perfect example for exploring PostgreSQL's strengths and limitations. Throughout this deep dive, we'll keep coming back to this example to ground our discussion in practical terms.

Core Capabilities & Limitations

With a motivating example in place, let's dive into what PostgreSQL can and can't do well. Most system design discussions about PostgreSQL will center around its read performance, write capabilities, consistency guarantees, and schema flexibility. Understanding these core characteristics will help you make informed decisions about when to use PostgreSQL in your design.

Read Performance

First up is read performance - this is critical because in most applications, reads vastly outnumber writes. In our social media example, users spend far more time browsing posts and profiles than they do creating content.

> In system design interviews, you don't need to dive into query planner internals. Instead, focus on practical performance patterns and when different types of indexes make sense.

When a user views a profile, we need to efficiently fetch all posts by that user. Without proper indexing, PostgreSQL would need to scan every row in the posts table to find matching posts - a process that gets increasingly expensive as our data grows. This is where indexes come in. By creating an index on the `user_id` column of our posts table, we can quickly locate all posts for a given user without scanning the entire table.

Basic Indexing

The most fundamental way to speed up reads in PostgreSQL is through indexes. By default, PostgreSQL uses B-tree indexes, which work great for:

- Exact matches (WHERE email = '**user@example.com**')
- Range queries (WHERE created_at > '2024-01-01')
- Sorting (ORDER BY username if the ORDER BY column match the index columns' order)

By default, PostgreSQL will create a B-tree index on your primary key column, but you also have the ability to create indexes on other columns as well.

```
-- This is your bread and butter index
CREATE INDEX idx_users_email ON users(email);

-- Multi-column indexes for common query patterns
CREATE INDEX idx_posts_user_date ON posts(user_id, created_at);
```

> A common trap in interviews is to suggest adding indexes for every column. Remember that each index:
>
> - Makes writes slower (as the index must be updated)
> - Takes up disk space
> - May not even be used if the query planner thinks a sequential scan would be faster

Beyond Basic Indexes

Where PostgreSQL really shines is its support for specialized indexes. These come up frequently in system design interviews because they can eliminate the need for separate specialized databases:

Full-Text Search using GIN indexes. Postgres supports full-text search out of the box using GIN (Generalized Inverted Index) indexes. GIN indexes work like the index at the back of a book - they store a mapping of each word to all the locations where it appears. This makes them perfect for full-text search where you need to quickly find documents containing specific words:

```
-- Add a tsvector column for search
ALTER TABLE posts ADD COLUMN search_vector tsvector;
CREATE INDEX idx_posts_search ON posts USING GIN(search_vector);

-- Now you can do full-text search
SELECT * FROM posts
WHERE search_vector @@ to_tsquery('postgresql & database');
```

For many applications, this built-in search capability means you don't need a separate **Elasticsearch** cluster. It supports everything from:

1. Word stemming (finding/find/finds all match)
2. Relevance ranking
3. Multiple languages
4. Complex queries with AND/OR/NOT

> While PostgreSQL's full-text search is powerful, it may not fully replace Elasticsearch for all use cases. Consider Elasticsearch when you need:
>
> - More sophisticated relevancy scoring
> - Faceted search capabilities
> - Fuzzy matching and "search as you type" features
> - Distributed search across very large datasets
> - Advanced analytics and aggregations

> Start with PostgreSQL's built-in search for simpler use cases. Only introduce Elasticsearch when you have specific requirements that PostgreSQL's search capabilities can't meet. This keeps your architecture simpler and reduces operational complexity.

JSONB columns with GIN indexes are particularly useful when you need flexible metadata on your posts. For example, in our social media platform, each post might have different attributes like location, mentioned users, hashtags, or attached media. Rather than creating separate columns for each possibility, we can store this in a JSONB column (giving us the flexibility to add new attributes as needed just like we would in a NoSQL database!).

```
-- Add a JSONB column for post metadata
ALTER TABLE posts ADD COLUMN metadata JSONB;
CREATE INDEX idx_posts_metadata ON posts USING GIN(metadata);

-- Now we can efficiently query posts with specific metadata
SELECT * FROM posts
WHERE metadata @> '{"type": "video"}'
  AND metadata @> '{"hashtags": ["coding"]}';

-- Or find all posts that mention a specific user
SELECT * FROM posts
WHERE metadata @> '{"mentions": ["user123"]}';
```

Geospatial Search with PostGIS. While not built into PostgreSQL core, the PostGIS extension adds powerful spatial capabilities. Just like we can index text for fast searching, PostGIS lets us index location data for efficient geospatial queries. This is perfect for our social media platform when we want to show users posts from their local area:

```
-- Enable PostGIS
CREATE EXTENSION postgis;

-- Add a location column to posts
ALTER TABLE posts
ADD COLUMN location geometry(Point);

-- Create a spatial index
CREATE INDEX idx_posts_location
ON posts USING GIST(location);

-- Find all posts within 5km of a user
SELECT * FROM posts
WHERE ST_DWithin(
    location::geography,
    ST_MakePoint(-122.4194, 37.7749)::geography, -- SF coordinates
    5000  -- 5km in meters
);
```

PostGIS is incredibly powerful - it can handle:

- Different types of spatial data (points, lines, polygons)
- Various distance calculations (as-the-crow-flies, driving distance)
- Spatial operations (intersections, containment)
- Different coordinate systems

> PostGIS is so capable that companies like Uber initially used it for their entire ride-matching system. While they've since moved to custom solutions for scale, it shows how far you can go with PostgreSQL's extensions before needing specialized databases.

The index type used here (GIST) is specifically optimized for geometric data, using R-tree indexing under the hood. This means queries like "find all posts within X kilometers" or "find posts inside this boundary" can be executed efficiently without having to check every single row.

> Just like with full-text search, you should consider PostGIS before reaching for a specialized geospatial database. It's another example of getting sophisticated functionality while keeping your architecture simple.

Better yet, we can combine all these capabilities to create rich search experiences. For example, we can find all video posts within 5km of San Francisco that mention "food" in their content and are tagged with "restaurant":

```
SELECT * FROM posts
WHERE search_vector @@ to_tsquery('food')
  AND metadata @> '{"type": "video", "hashtags": ["restaurant"]}'
  AND ST_DWithin(
    location::geography,
    ST_MakePoint(-122.4194, 37.7749)::geography,
    5000
  );
```

Let me rewrite this section to better explain the why and connect the concepts:

Query Optimization Essentials

So far we've covered the different types of indexes PostgreSQL offers, but there's more to query optimization than just picking the right index type. Let's look at some advanced indexing strategies that can dramatically improve read performance.

Covering Indexes

When PostgreSQL uses an index to find a row, it typically needs to do two things:

1. Look up the value in the index to find the row's location
2. Fetch the actual row from the table to get other columns you need

But what if we could store all the data we need right in the index itself? That's what covering indexes do:

```
-- Let's say this is a common query in our social media app:
SELECT title, created_at
FROM posts
WHERE user_id = 123
ORDER BY created_at DESC;

-- A covering index that includes all needed columns
CREATE INDEX idx_posts_user_include
ON posts(user_id) INCLUDE (title, created_at);
```

> Covering indexes can make queries significantly faster because PostgreSQL can satisfy the entire query just from the index without touching the table. The trade-off is that the index takes up more space and writes become slightly slower.

Partial Indexes

Sometimes you only need to index a subset of your data. For example, in our social media platform, most queries are probably looking for active users, not deleted ones:

```
-- Standard index indexes everything
CREATE INDEX idx_users_email ON users(email);  -- Indexes ALL users

-- Partial index only indexes active users
CREATE INDEX idx_active_users
ON users(email) WHERE status = 'active';  -- Smaller, faster index
```

Partial indexes are particularly effective in scenarios where most of your queries only need a subset of rows, when you have many "inactive" or "deleted" records that don't need to be indexed, or when you want to reduce the overall size and maintenance overhead of your indexes. By only indexing the relevant subset of data, partial indexes can significantly improve both query performance and resource utilization.

Practical Performance Limits

There is a good chance that during your non-functional requirements you outlined some latency goals. Ideally, you even quantified them! That means that as you go deep into the design, you need some basic performance numbers in mind. These numbers are very rough estimates as real numbers depend heavily on the hardware and the specific workload. That said, estimates should be enough to get you started in an interview.

1. **Query Performance**:
   - Simple indexed lookups: tens of thousands per second per core
   - Complex joins: thousands per second
   - Full-table scans: depends heavily on whether data fits in memory
2. **Scale Limits**:
   - Tables start getting unwieldy past 100M rows
   - Full-text search works well up to tens of millions of documents
   - Complex joins become challenging with tables >10M rows
   - Performance drops significantly when working set exceeds available RAM

> These aren't hard limits - PostgreSQL can handle much more with proper optimization. But they're good rules of thumb for when you should start considering partitioning, sharding, or other scaling strategies.

Keep in mind, memory is king when it comes to performance! Queries that can be satisfied from memory are orders of magnitude faster than those requiring disk access. As a rule of thumb, you should try to keep your working set (frequently accessed data) in RAM for optimal performance.

> In your interview, showing knowledge of these practical limits helps demonstrate that you understand not just how to use PostgreSQL, but when you might need to consider alternatives or additional optimization strategies.

Write Performance

Now let's talk about writes. While reads might dominate most workloads, write performance is often more critical because it directly impacts user experience - nobody wants to wait seconds after hitting "Post" for their content to appear.

When a write occurs in PostgreSQL, several steps happen to ensure both performance and durability:

1. **Transaction Log (WAL) Write [Disk]**: Changes are first written to the Write-Ahead Log (WAL) on disk. This is a sequential write operation, making it relatively fast. The WAL is critical for durability - once changes are written here, the transaction is considered durable because even if the server crashes, PostgreSQL can recover the changes from the WAL.
2. **Buffer Cache Update [Memory]**: Changes are made to the data pages in PostgreSQL's shared buffer cache, where the actual tables and indexes live in memory. When pages are modified, they're marked as "dirty" to indicate they need to be written to disk eventually.
3. **Background Writer [Memory → Disk]**: Dirty pages in memory are periodically written to the actual data files on disk. This happens asynchronously through the background writer, when memory pressure gets too high, or when a checkpoint occurs. This delayed write strategy allows PostgreSQL to batch multiple changes together for better performance.
4. **Index Updates [Memory & Disk]**: Each index needs to be updated to reflect the changes. Like table data, index changes also go through the WAL for durability. This is why having many indexes can significantly slow down writes - each index requires additional WAL entries and memory updates.

> This architecture is why PostgreSQL can be fast for writes - most of the work happens in memory, while ensuring durability through the WAL. The actual writing of data pages to disk happens later and is optimized for batch operations.

The practical implication of this design is that write performance is typically bounded by how fast you can write to the WAL (disk I/O), how many indexes need to be updated, and how much memory is available for the buffer cache.

Throughput Limitations

Now we know about what happens when a write occurs in PostgreSQL, before we go onto optimizations, let's first talk about the practical limits of write throughput. This is important to know as it will help you decide whether PostgreSQL is a good fit for your system.

A well-tuned PostgreSQL instance on good (not great) hardware can handle:

- Simple inserts: ~5,000 per second per core
- Updates with index modifications: ~1,000-2,000 per second per core
- Complex transactions (multiple tables/indexes): Hundreds per second
- Bulk operations: Tens of thousands of rows per second

> These numbers assume PostgreSQL's default transaction isolation level (Read Committed), where transactions only see data that was committed before their query began. If you change the default isolation level these numbers can go up or down.

What affects these limits? Several factors:

- Hardware: Write throughput is often bottlenecked by disk I/O for the WAL
- Indexes: Each additional index reduces write throughput
- Replication: If configured, synchronous replication adds latency as we wait for replicas to confirm
- Transaction Complexity: More tables or indexes touched = slower transactions

> Remember, were talking about a single node here! So if your system has higher write throughput that, say, 5k writes per second, this does not mean that PostgreSQL is off the table, it just means that you are going to need to shard your data across multiple nodes/machines.

Let me continue from there, building on our social media example:

Write Performance Optimizations

Ok, a single node can handle around 5k writes per second, so what can we do? How can we improve our write performance if we require more than that? Let's look at strategies ranging from simple optimizations to architectural changes.

We have a few options:

1. Batch Processing
2. Vertical Scaling
3. Write Offloading
4. Table Partitioning
5. Sharding

Let's discuss each of these in turn.

1. Vertical Scaling

Before jumping to complex solutions, we can always consider just upgrading our hardware. This could mean using faster NVMe disks for better WAL performance, adding more RAM to increase the buffer cache size, or upgrading to CPUs with more cores to handle parallel operations more effectively.

This usually isn't the most compelling solution in an interview, but it's a good place to start.

2. Batch Processing

The simplest optimization is to batch writes together. Instead of processing each write individually, we collect multiple operations and execute them in a single transaction. For example, instead of inserting 1000 likes one at a time, we can insert them all in a single transaction. This means we're buffering writes in our server's memory before committing them to disk. The risk here is clear, if we crash in the middle of a batch, we'll lose all the writes in that batch.

```
-- Instead of 1000 separate inserts:
INSERT INTO likes (post_id, user_id) VALUES
  (1, 101), (1, 102), ..., (1, 1000);
```

3. Write Offloading

Some writes don't need to happen synchronously. For example, analytics data, activity logs, or aggregated metrics can often be processed asynchronously. Instead of writing directly to PostgreSQL, we can:

1. Send writes to a message queue (like Kafka)
2. Have background workers process these queued writes in batches
3. Optionally maintain a separate analytics database

This pattern works especially well for handling activity logging, analytics events, metrics aggregation, and non-critical updates like "last seen" timestamps. These types of writes don't need to happen immediately and can be processed in the background without impacting the core user experience.

4. Table Partitioning

For large tables, partitioning can improve both read and write performance by splitting data across multiple physical tables. The most common use case is time-based partitioning. Going back to our social media example, let's say we have a posts table that grows by millions of rows per month:

```
CREATE TABLE posts (
    id SERIAL,
    user_id INT,
    content TEXT,
    created_at TIMESTAMP
) PARTITION BY RANGE (created_at);

-- Create partitions by month
CREATE TABLE posts_2024_01 PARTITION OF posts
    FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');
```

Why does this help writes? First, different database sessions can write to different partitions simultaneously, increasing concurrency. Second, when data is inserted, index updates only need to happen on the relevant partition rather than the entire table. Finally, bulk loading operations can be performed partition by partition, making it easier to load large amounts of data efficiently.

Conveniently, it also helps with reads. When users view recent posts, PostgreSQL only needs to scan the recent partitions. No need to wade through years of historical data.

> A common pattern is to keep recent partitions on fast storage (like NVMe drives) while moving older partitions to cheaper storage. Users get fast access to recent data, which is typically what they care about most.

This is a great strategy for data that has a natural lifecycle like social media posts where new ones are most relevant.

5. Sharding

This is the most common solution in an interview. When a single node isn't enough, sharding lets you distribute writes across multiple PostgreSQL instances. You'll just want to be clear about what you're sharding on and how you're distributing the data.

For example, we may consider sharing our posts by `user_id`. This way, all the data for a user lives on a single shard. This is important, because when we go to read the data we want to avoid cross-shard queries where we need to scatter-gather data from multiple shards.

You typically want to shard on the column that you're querying by most often. So if we typically query for all posts from a given user, we'll shard by `user_id`.

> Sharding adds complexity - you'll need to handle cross-shard queries, maintain consistent schemas across shards, and manage multiple databases. Only introduce it when simpler optimizations aren't sufficient.

Unlike DynamoDB, PostgreSQL doesn't have a built-in sharding solution. You'll need to implement sharding manually, which can be a bit of a challenge. Alternatively, you can use managed services like **Citus** which handles many of the sharding complexities for you.

Replication

While we've discussed how to optimize write performance on a single node, most real-world deployments use replication for two key purposes:

1. Scaling reads by distributing queries across replicas
2. Providing high availability in case of node failures

Replication is the process of copying data from one database to one or more other databases. This is a key part of PostgreSQL's scalability and availability story.

PostgreSQL supports two main types of replication: synchronous and asynchronous. In synchronous replication, the primary waits for acknowledgment from replicas before confirming the write to the client. With asynchronous replication, the primary confirms the write to the client immediately and replicates changes to replicas in the background. While the technical details may not come up in an interview, understanding these tradeoffs is important - synchronous replication provides stronger consistency but higher latency, while asynchronous replication offers better performance but potential inconsistency between replicas.

> Many organizations use a hybrid approach: keeping a small number of synchronous replicas for stronger consistency while maintaining additional asynchronous replicas for read scaling. PostgreSQL allows you to specify which replicas should be synchronous.

Scaling Reads

The most common use for replication is to scale read performance. By creating read replicas, you can distribute read queries across multiple database instances while sending all writes to the primary. This is particularly effective because most applications are read-heavy.

Let's go back to our social media example. When users browse their feed or view profiles, these are all read operations that can be handled by any replica. Only when they create posts or update their profile do we need to use the primary. Now we have multiplied our read throughput by N where N is the number of replicas.

> There's one key caveat with read replicas: replication lag. If a user makes a change and immediately tries to read it back, they might not see their change if they hit a replica that hasn't caught up yet. This is known as "read-your-writes" consistency.

High Availability

The second major benefit of replication is high availability. By maintaining copies of your data across multiple nodes, you can handle hardware failures without downtime. If your primary node fails, one of the replicas can be promoted to become the new primary.

This failover process typically involves:

1. Detecting that the primary is down
2. Promoting a replica to primary
3. Updating connection information
4. Repointing applications to the new primary

> In your interview, emphasize that replication isn't just about scaling - it's about reliability. You might say: "We'll use replication not just for distributing read load, but also to ensure our service stays available even if we lose a database node."

Most teams use managed PostgreSQL services (like AWS RDS or GCP Cloud SQL) that handle the complexities of failover automatically. In your interview, it's enough to know that failover is possible and roughly how it works - you don't need to get into the details of how to configure it manually.

Data Consistency

If you've chosen to prioritize consistency over availability in your non-functional requirements, then PostgreSQL is a strong choice. It's built from the ground up to provide strong consistency guarantees through ACID transactions. However, simply choosing PostgreSQL isn't enough - you need to understand how to actually achieve the consistency your system requires.

> A common mistake in interviews is to say "We'll use PostgreSQL because it's ACID compliant" without being able to explain how you'll actually use those ACID properties to solve your consistency requirements.

Transactions

One of the most common points of discussion in interviews ends up being around transactions. A transaction is a set of operations that are executed together and must either all succeed or all fail together. This is the foundation for ensuring consistency in PostgreSQL.

Let's consider a simple example where we need to transfer money between two bank accounts. We need to ensure that if we deduct money from one account, it must be added to the other account. Neither operation can happen in isolation:

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;
```

This transaction ensures atomicity - either both updates happen or neither does. However, transactions alone don't ensure consistency in all scenarios, particularly when multiple transactions are happening concurrently.

Transactions and Concurrent Operations

Transactions ensure consistency for a single series of operations, but things get more complicated when multiple transactions are happening concurrently. Remember, in most real applications, you'll have multiple users or services trying to read and modify data at the same time.

This is where many candidates get tripped up in interviews. They understand basic transactions but haven't thought through how to maintain consistency when multiple operations are happening simultaneously.

Let's look at an auction system as an example. Here users place bids on items and we accept bids only if they're higher than the current max bid. A single transaction can ensure that checking the current bid and placing a new bid happen atomically, but what happens when two users try to bid at the same time?

```
BEGIN;
-- Get current max bid for item 123
SELECT maxBid from Auction where id = 123;

-- Place new bid if it's higher
INSERT INTO bids (item_id, user_id, amount)
VALUES (123, 456, 100);

-- Update the max bid
UPDATE Auction SET maxBid = 100 WHERE id = 123;
COMMIT;
```

> Even though this is in a transaction, with PostgreSQL's default isolation level (Read Committed), we could still have consistency problems if two users bid simultaneously. Both transactions could read the same max bid before either commits.

Here's how this could lead to an inconsistent state:

1. User A's transaction reads current max bid: $90
2. User B's transaction reads current max bid: $90
3. User A places bid for $100
4. User A commits
5. User B places bid for $95
6. User B commits

Now we have an invalid state: a $95 bid was accepted after a $100 bid!

There are two main ways we can solve this concurrency issue:

1. Row-Level Locking

The simplest solution is to lock the auction row while we're checking and updating bids. By using the `FOR UPDATE` clause, we tell PostgreSQL to lock the rows we're reading. Other transactions trying to read these rows with `FOR UPDATE` will have to wait until our transaction completes. This ensures we have a consistent view of the data while making our changes.

```
BEGIN;
-- Lock the item and get current max bid
SELECT maxBid FROM Auction WHERE id = 123 FOR UPDATE;

-- Place new bid if it's higher
INSERT INTO bids (item_id, user_id, amount)
VALUES (123, 456, 100);

-- Update the max bid
UPDATE Auction SET maxBid = 100 WHERE id = 123;
COMMIT;
```

So when you need to ensure that two operations happen atomically, you'll want to emphasize to your interviewer how you will achieve that beyond simply saying "we'll use transactions". Instead, "we'll use transactions and row-level locking on the auction row", for this case.

2. Higher Isolation Level

Alternatively, we can use a stricter isolation level:

```
BEGIN;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

-- Same code as before...

COMMIT;
```

> While serializable isolation prevents all consistency anomalies, it comes with a cost: if two transactions conflict, one will be rolled back and need to retry. Your application needs to be prepared to handle these retry scenarios.

PostgreSQL supports three isolation levels, each providing different consistency guarantees:

1. **Read Committed** (Default) is PostgreSQL's default isolation level that only sees data that was committed before the query began. As transactions execute, each query within a transaction can see new commits made by other transactions that completed after the transaction started. While this provides good performance, it can lead to non-repeatable reads where the same query returns different results within a transaction.
2. **Repeatable Read** in PostgreSQL provides stronger guarantees than the SQL standard requires. It creates a consistent snapshot of the data as of the start of the transaction, and unlike other databases, PostgreSQL's implementation prevents both non-repeatable reads AND phantom reads. This means not only will the same query return the same results within a transaction, but no new rows will appear that match your query conditions - even if other transactions commit such rows.
3. **Serializable** is the strongest isolation level that makes transactions behave as if they were executed one after another in sequence. This prevents all types of concurrency anomalies but comes with the trade-off of requiring retry logic in your application to handle transaction conflicts.

> PostgreSQL's implementation of Repeatable Read is notably stronger than what the SQL standard requires. While other databases might allow phantom reads at this isolation level, PostgreSQL prevents them. This means you might not need Serializable isolation in cases where you would in other databases.

So, when should you use row-locking and when should you use a higher isolation level?

Aspect	Serializable Isolation	Row-Level Locking
Concurrency	Lower - transactions might need to retry on conflict	Higher - only conflicts when touching same rows
Performance	More overhead - must track all read/write dependencies	Less overhead - only locks specific rows
Use Case	Complex transactions where it's hard to know what to lock	When you know exactly which rows need atomic updates
Complexity	Simple to implement but requires retry logic	More explicit in code but no retries needed
Error Handling	Must handle serialization failures	Must handle deadlock scenarios
Example	Complex financial calculations across multiple tables	Auction bidding, inventory updates
Memory Usage	Higher - tracks entire transaction history	Lower - only tracks locks
Scalability	Doesn't scale as well with concurrent transactions	Scales better when conflicts are rare

> Row-level locking is generally preferred when you know exactly which rows need to be locked. Save serializable isolation for cases where the transaction is too complex to reason about which locks are needed.

When to Use PostgreSQL (and When Not To)

Let me revise that opening with more concrete technical advantages:

Here's my advice: in your system design interview, PostgreSQL should be your default choice unless you have a specific reason to use something else. Why? Because PostgreSQL:

1. Provides strong ACID guarantees while still scaling effectively with replication and partitioning
2. Handles both structured and unstructured data through JSONB support
3. Includes built-in solutions for common needs like full-text search and geospatial queries
4. Can scale reads effectively through replication
5. Offers excellent tooling and a mature ecosystem

> Start with PostgreSQL, then justify why you might need to deviate. This is much stronger than starting with a niche solution and trying to justify why it's better than PostgreSQL.

PostgreSQL shines when you need:

- Complex relationships between data
- Strong consistency guarantees
- Rich querying capabilities
- A mix of structured and unstructured data (JSONB)
- Built-in full-text search
- Geospatial queries

For example, it's perfect for:

- E-commerce platforms (inventory, orders, user data)
- Financial systems (transactions, accounts, audit logs)
- Content management systems (posts, comments, users)
- Analytics platforms (up to reasonable scale)

When to Consider Alternatives

That said, we aren't maxis over here. There are legitimate reasons to look beyond PostgreSQL.

1. Extreme Write Throughput

If you need to handle millions of writes per second, PostgreSQL will struggle because each write requires a WAL entry and index updates, creating I/O bottlenecks even with the fastest storage. Even with sharding, coordinating writes across many PostgreSQL nodes adds complexity and latency. In these cases, you might consider:

- NoSQL databases (like **Cassandra**) for event streaming
- Key-value stores (like **Redis**) for real-time counters

2. Global Multi-Region Requirements

When you need active-active deployment across regions (where multiple regions accept writes simultaneously), PostgreSQL faces fundamental limitations. Its single-primary architecture means one region must be designated as the primary writer, while others act as read replicas. Attempting true active-active deployment creates significant challenges around data consistency and conflict resolution, as PostgreSQL wasn't designed to handle simultaneous writes from multiple primaries. The synchronous replication needed across regions also introduces substantial latency, as changes must be confirmed by distant replicas before being committed. For these scenarios, consider:

- CockroachDB for global ACID compliance
- **Cassandra** for eventual consistency at global scale
- **DynamoDB** for managed global tables

3. Simple Key-Value Access Patterns

If your access patterns are truly key-value (meaning you're just storing and retrieving values by key without joins or complex queries), PostgreSQL is overkill. Its MVCC architecture, WAL logging, and complex query planner add overhead you don't need. In these cases, consider:

- **Redis** for in-memory performance
- **DynamoDB** for managed scalability
- **Cassandra** for write-heavy workloads

> Scalability alone is not a good reason to choose an alternative to PostgreSQL. PostgreSQL can handle significant scale with proper design.

Summary

PostgreSQL should be your default choice in system design interviews unless specific requirements demand otherwise. Its combination of ACID compliance, rich feature set, and scalability options make it suitable for a wide range of use cases, from simple CRUD applications to complex transactional systems.

When discussing PostgreSQL in interviews, focus on analyzing concrete requirements around data consistency, query patterns, and scale, rather than following trends. Be prepared to discuss key trade-offs like ACID vs eventual consistency, read vs write scaling strategies, and indexing decisions. Start simple and add complexity only as needed.

PostgreSQL's rich feature set often eliminates the need for additional systems in your architecture. Its full-text search capabilities might replace Elasticsearch, JSONB support could eliminate the need for MongoDB, and PostGIS handles geospatial needs that might otherwise require specialized databases. Built-in replication often provides sufficient scaling capabilities for many use cases. However, it's equally important to recognize when PostgreSQL might not be the best fit, such as cases requiring extreme write scaling or global distribution, where databases like Cassandra or CockroachDB might be more appropriate.

Appendix: Basic SQL Concepts

Before diving into PostgreSQL-specific features, let's review how SQL databases organize data. These core concepts apply to any SQL database, not just PostgreSQL but are a necessary foundation that we will build on throughout this deep dive.

Relational Database Principles

At its core, PostgreSQL stores data in tables (also called relations). Think of a table like a spreadsheet with rows and columns. Each column has a specific data type (like text, numbers, or dates), and each row represents one complete record.

Let's look at a concrete example. Imagine we're designing a social media platform. We might have a `users` table that looks like this:

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

This command would create the following table (data is just for example):

id	username	email	created_at
1	johndoe	**john@example.com**	2024-01-01 10:00:00
2	janedoe	**jane@example.com**	2024-01-01 10:05:00
3	bobsmith	**bob@example.com**	2024-01-01 10:10:00

When a new user signs up, we create a new row in this table. Each user gets a unique `id` (that's what `PRIMARY KEY` means), and we ensure no two users can have the same username or email (that's what `UNIQUE` does).

But users aren't much fun by themselves. They need to be able to post content. Here's where the "relational" part of relational databases comes in. We can create a `posts` table that's connected to our users:

```
CREATE TABLE posts (
    id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(id),
    content TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

id	user_id	content	created_at
1	1	Hello, world!	2024-01-01 10:00:00
2	1	My first post	2024-01-01 10:05:00
3	2	Another post	2024-01-01 10:10:00

See `REFERENCES users(id)` ? That's called a foreign key - it creates a relationship between posts and users. Every post must belong to a valid user, and PostgreSQL will enforce this for us. This is one of the key strengths of relational databases: they help maintain data integrity by enforcing these relationships.

> In your interview, being able to explain these relationships is crucial. There are three main types:
>
> - One-to-One: Like a user and their profile settings
> - One-to-Many: Like our users and posts (one user can have many posts)
> - Many-to-Many: Like users and the posts they like (which we'll see next)

Now, what if we want users to be able to like posts? This introduces a many-to-many relationship - one user can like many posts, and one post can be liked by many users. We handle this with what's called a join table:

```
CREATE TABLE likes (
    user_id INTEGER REFERENCES users(id),
    post_id INTEGER REFERENCES posts(id),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (user_id, post_id)
);
```

This structure, where we break data into separate tables and connect them through relationships, is called "normalization." It helps us:

1. Avoid duplicating data (we don't store user information in every post)
2. Maintain data integrity (if a user changes their username, it updates everywhere)
3. Make our data model flexible (we can add new user attributes without touching posts)

> While normalization is generally good, sometimes we intentionally denormalize data for performance. For example, we might store a post's like count directly in the posts table even though we could calculate it from the likes table. This trade-off between data consistency and query performance is exactly the kind of thing you should discuss in your interview!

Understanding these fundamentals - tables, relationships, and normalization - is important. They're what make SQL databases like PostgreSQL so powerful for applications that need to maintain complex relationships between different types of data. In your interview, being able to explain not just how to use these concepts, but when and why to use them (or break them), will set you apart.

ACID Properties

One of PostgreSQL's greatest strengths is its strict adherence to ACID (Atomicity, Consistency, Isolation, and Durability) properties. If you've used databases like MongoDB or Cassandra, you're familiar with eventual consistency or relaxed transaction guarantees which are common trade-offs in NoSQL databases. PostgreSQL takes a different approach – it ensures that your data always follows all defined rules and constraints (like foreign keys, unique constraints, and custom checks), and that all transactions complete fully or not at all, even if it means sacrificing some performance.

Let's break down what ACID means using a real-world example: transferring money between bank accounts.

Atomicity (All Or Nothing)

Imagine you're transferring $100 from your savings to your checking account. This involves two operations:

1. Deduct $100 from savings
2. Add $100 to checking

```
BEGIN;
  UPDATE accounts SET balance = balance - 100 WHERE account_id = 'savings';
  UPDATE accounts SET balance = balance + 100 WHERE account_id = 'checking';
COMMIT;
```

Atomicity guarantees that either both operations succeed or neither does. If the system crashes after deducting from savings but before adding to checking, PostgreSQL will roll back the entire transaction. Your money never disappears into thin air.

> In your interview, emphasize how atomicity prevents partial failures. Without it, distributed systems can end up in inconsistent states that are very difficult to recover from.

Consistency (Data Integrity)

Consistency ensures that transactions can only bring the database from one valid state to another. For example, let's say we have a rule that account balances can't go negative:

```
CREATE TABLE accounts (
    account_id TEXT PRIMARY KEY,
    balance DECIMAL CHECK (balance >= 0),
    owner_id INTEGER REFERENCES users(id)
);
```

If a transaction would make your balance negative, PostgreSQL will reject the entire transaction. This is different from NoSQL databases where you often have to enforce these rules in your application code.

> Confusingly, consistency in ACID has a slightly different meaning than consistency in CAP Theorem. In ACID, consistency means that the database always follows all defined rules and constraints. In the CAP Theorem, consistency means that the database always returns the correct result, even if it means sacrificing availability or partition tolerance.

Isolation (Concurrent Transactions)

Isolation levels determine how transactions can interact with data that's being modified by other concurrent transactions. PostgreSQL supports four isolation levels, each preventing different types of phenomena:

```
BEGIN;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  -- Default level
-- or REPEATABLE READ
-- or SERIALIZABLE
COMMIT;
```

> PostgreSQL's default "Read Committed" level only prevents dirty reads (reading uncommitted changes). It allows non-repeatable reads and phantom reads. Check the **PostgreSQL documentation** for a detailed breakdown of which anomalies each level prevents.

While the SQL standard defines four isolation levels, PostgreSQL implements only three distinct levels internally. Specifically, Read Uncommitted behaves identically to Read Committed in PostgreSQL. This design choice aligns with PostgreSQL's multiversion concurrency control (MVCC) architecture, which always provides snapshot isolation - making it impossible to read uncommitted data.

Isolation Level	Dirty Read	Nonrepeatable Read	Phantom Read	Serialization Anomaly
Read uncommitted	Allowed, but not in PG	Possible	Possible	Possible
Read committed	Not possible	Possible	Possible	Possible
Repeatable read	Not possible	Not possible	Allowed, but not in PG	Possible
Serializable	Not possible	Not possible	Not possible	Not possible

Durability (Permanent Storage)

Once PostgreSQL says a transaction is committed, that data is guaranteed to have been written to disk and sync'd, protecting against crashes or power failures. This is achieved through Write-Ahead Logging (WAL):

1. Changes are first written to a log
2. The log is flushed to disk
3. Only then is the transaction considered committed

> While durability is guaranteed, there's a performance cost. Some applications might choose to relax durability for speed (like setting `synchronous_commit = off`), meaning some writes which haven't been written to disk may be lost in the event of a power outage or crash.

Why ACID Matters

In your interview, you'll often need to decide between different types of databases. ACID compliance is a crucial factor in this decision. Consider these scenarios:

- **Financial transactions**: You absolutely need ACID properties to prevent money from being lost or double-spent
- **Social media likes**: You might be okay with eventual consistency
- **User authentication**: You probably want ACID to prevent security issues
- **Analytics data**: You might prioritize performance over strict consistency

PostgreSQL's strict ACID compliance makes it an excellent choice for systems where data consistency is crucial. While performance trade-offs nowadays are minor, they're still worth being aware of.

SQL Language

Let's talk about SQL briefly. While you rarely write actual SQL queries in system design interviews, understanding SQL's capabilities helps you make better architectural decisions. Plus, if you're interviewing for a more junior role, you might be asked to write some basic queries to demonstrate database understanding.

SQL Command Types

SQL commands fall into four main categories:

1. **DDL (Data Definition Language)**
   - Creates and modifies database structure
   - Examples: `CREATE TABLE` , `ALTER TABLE` , `DROP TABLE`

```
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  email VARCHAR(255) UNIQUE
);

ALTER TABLE users ADD COLUMN username TEXT;
```

2. **DML (Data Manipulation Language)**
   - Manages data within tables
   - Examples: `SELECT` , `INSERT` , `UPDATE` , `DELETE`

```
-- Find all users who joined in the last week
SELECT * FROM users
WHERE created_at > NOW() - INTERVAL '7 days';

-- Update a user's email
UPDATE users SET email = 'new@email.com'
WHERE id = 123;
```

3. **DCL (Data Control Language)**
   - Controls access permissions
   - Examples: `GRANT` , `REVOKE`

```
-- Give read access to a specific user
GRANT SELECT ON users TO read_only_user;
```

4. **TCL (Transaction Control Language)**
   - Manages transactions
   - Examples: `BEGIN` , `COMMIT` , `ROLLBACK`

```
BEGIN;
  -- Multiple operations...
COMMIT;
```

> In your interview, you might be asked about database access patterns rather than specific queries. For example, "How would you query this data efficiently?" or "What indexes would you create?" These questions test your understanding of database concepts rather than SQL syntax.

Test Your Knowledge

Take a quick 15 question quiz to test what you've learned.

Start Quiz

Mark as read

Add a comment...

Comment

Anonymous

Posting as leetcode

Search comments

Sort By Old

MinorBlueDove142 • 8 months ago

You mentioned the same for DynamoDB as default choice in System Design - b/w Postgres and Dynamo, which one can I chose by default?

1 Reply

Evan King • 8 months ago

I'll update the DDB article to ensure there is no confusion here. When in doubt, I'd lean towards Postgres. But the reality is, in many cases, there is no "wrong" answer here. It's about how you justify it.

In fact, "either works, but I X is what I have more experience with" is a reasonable justification, believe it or not. And it avoids the false dichotomy.

5 Reply

Evan King • 8 months ago

I actually don't see where I say to choose DDB w/ regards to a default choice. Did you have the specific line? Would love to get that updated.

4 Reply

MinorBlueDove142 • 8 months ago

this is from DDB article: "For system design interviews in particular, it has just about everything you'd ever need from a database. It even supports transactions now! Which neutralizes one of the biggest criticisms of DynamoDB in the past."

I think I misunderstood it.

1 Reply

QuietBlackJay631 • 7 months ago

Funnily enough re: "either works, but X is what I have more experience with", in a recent interview round I got feedback that this was undesirable to say even though everything I said before it was justifications for using DDB instead of Postgres...so I'd caution that some interviewers only hear 'I want to use this because I know it' even if you've provided justifications.

7 Reply

MathematicalPurpleAmphibian147 • 8 months ago

Thanks for putting together this guide - outcome focused and effective. Quick Question - For interview Qs that dont need ACID, why would engineering complexity (developer skill/training, DB Admins, lower flexibility of schema changes, high monitoring effort) compared to a managed solution like DDB not a point of trade-off?

3 Reply

Evan King • 8 months ago

Sounds like a totally valid trade-off to me!

What we're looking to avoid is statements like, "choose NoSQL because of scale," which show a lack of understanding of how far modern RDBMSs have come. But the list you have there are absolutely valid justifications.

2 Reply

Mike Choi • 8 months ago

Hey Evan correct me if I'm wrong, but when discussing DB tradeoffs we should really be focused around the application needs and choosing a DB that supports that use case right?

For instance, you could say you will use Postgres for everything (since it can pretty much handle most application use cases), but then explain as the system scales, you want to add something specific, say for analytical data (and point to a wide column/columnar DB). Or you wouldn't want to choose Mongo right out of the gate just "because its flexible" when a standard RDBMS will often times lead to less headache down the road (for devs just learning about DB management).

About your point on scalability, modern day RDBMS can scale to extremely high volume of data and throughput, but the challenge probably then comes in how you optimize the database with indexes, replicas, partitioning, etc., right? Albeit, scaling out with a RDBMS is probably more difficult than a NoSQL DB.

These are on top of other points, i.e. CAP theorem, if your system even needs a distributed database system right away, emphasis on read or write throughput, ACID properties, etc...

2 Reply

Evan King • 8 months ago

Yah, this is great. The key is just to show your interviewer that you understand nuance. This does not need to be a lot of information, but avoid the false SQL vs NoSQL dichotomy. Something as simple as, "Our data has XYZ property, so I'll opt for W because it handles this well and is what I know best," is a great answer.

0 Reply

ExtraAmaranthDuck389 • 3 months ago

That's awesome!

0 Reply

PersonalBrownFalcon225 • 8 months ago

How do we support atomic transactions if we have multiple nodes (I think we need something like 2 phase commit here?)

3 Reply

BeneficialBrownBarnacle904 • 8 months ago

Is there a recommendation on when a dedicated search optimized database like ElasticSearch would be needed vs when Postgres GIN/ PostGIS indexes would be sufficient?

1 Reply

Evan King • 8 months ago

Added a section for this! Good question. If you need:

- More sophisticated relevancy scoring
- Faceted search capabilities
- Fuzzy matching and "search as you type" features
- Real-time index updates
- Distributed search across very large datasets
- Advanced analytics and aggregations

Main one being if you're growing into the 100Ms of rows, postgres might not be the best fit for search.

1 Reply

BeneficialBrownBarnacle904 • 8 months ago

Thank you for the detailed answer and the bonus section! :D

0 Reply

RainyCyanHippopotamus256 • 8 months ago

"Main one being if you're growing into the 100Ms of rows, postgres might not be the best fit for search."

Why postgres might not be the best fit for many rows? Is it because ES can shard the data and horizontally scale the data automatically with ability to search through them by aggregating the data? But with postgres, it is generally a read-replica which could be a bottleneck?

1 Reply

Evan King • 8 months ago

PostgreSQL could technically can handle large-scale search through manual sharding, but you'd need to build your own logic to merge and rank results across shards which could be a pain. But, Elasticsearch was purpose-built for this. Automatic sharding and cross-node result aggregation are built into its core architecture.

1 Reply

RainyCyanHippopotamus256 • 8 months ago

Thanks. Yup, that's what I meant.

0 Reply

Eneias Silva • 4 months ago

Can you elaborate on Real-time index updates?

How is the Postgres index updated? Isn't it in real-time? Also Elastic Search index must be kept in sync with the main DB

0 Reply

Jiatang Dong • 8 months ago

"PostgreSQL supports two main types of replication: synchronous and asynchronous. Synchronous replication ensures that changes are written to disk on the primary node before being acknowledged, while asynchronous replication allows writes to complete on the primary node before being acknowledged."

I find this paragraph is confusing, can you explain more?

0 Reply

cst labs • 8 months ago

this is a typical replication setup. Imagine a write to the leader in synchronous replication. The response to the write is blocked until the replica doesn't acknowledge or the request times out. This leads to high consistency since every write is acknowledged. However, it leads to bad experience and affects availability. Async replication is actually async where the leader won't block for an ack from the replica. Though, an organization may choose to have a limited number of synchronous and rest async replication. A consensus algorithm is used for this purpose with a defined quorum.

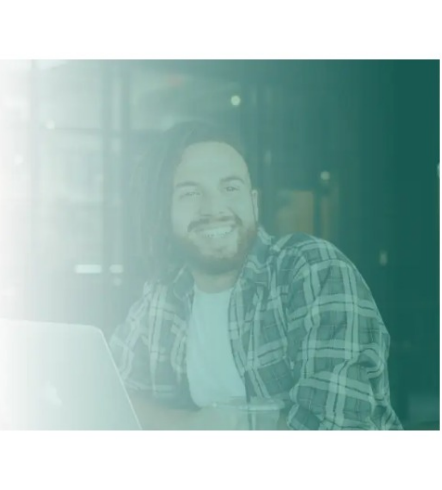
0 Reply

RadicalBlackBeetle554 • 8 months ago

I think, there must be some typo. Evan, could you please take a re-look in that paragraph (synchronous vs asynchronous replication) and clarify.

0 Reply

Show All Comments



Schedule a mock interview

Meet with a FAANG senior+ engineer or manager and learn exactly what it takes to get the job.

Schedule A Mock Interview

Questions
Meta SWE Interview Questions
Amazon SWE Interview Questions
Google SWE Interview Questions
Engineering Manager (EM) Interview Questions

Learn
Learn System Design
Learn DSA
Learn Behavioral
Learn ML System Design
Guided Practice

Links
FAQ
Pricing
Gift Mock Interviews
Gift Premium
Become a Coach
Our Coaches
Employers
Hello Interview Premium

Legal
Terms and Conditions
Privacy Policy

Contact
About Us
support@hellointerview.com
7511 Greenwood Ave North
Unit #4238 Seattle
WA 98103

© 2025 Optick Labs Inc. All rights reserved.